

RankCloak Conceals the Surface Form of Synthetic Cryptographic Artifacts in Language Model Generated Text

Alexander V. Mantzaris^{1,*}

¹School of Data, Mathematical, and Statistical Sciences, University of Central Florida, Orlando, FL, United States of America

*Correspondence: alexander.mantzaris@ucf.edu

ABSTRACT

Cryptographic artifacts such as hashes, nonces and ciphertexts are conspicuous in their usual textual forms. RankCloak represents synthetic artifacts through language model token choices that conceal their original syntax. We evaluated 480 deterministic artifacts from eight classes with three model families and 18 English prompts. All 6,480 primary trials recovered the original bytes when token sequences and configurations were preserved. Visible-text retokenization recovered 88 of 144 covers. The tested blockquote and trimming operation, paraphrasing and cross-model decoding failed. Strong detector performance after deduplication, including under generating-model trace access, limits stealth. Entropy gating reduced some detection signals but lowered serialized-payload rates and left six strict-gate payloads incomplete. In a contextual follow-up with blinded model judges, 537 of 576 message pairs had complete ratings. Under a rubric allowing minor awkwardness, acceptance averaged 90.3% for RankCloak and 96.6% for matched greedy ordinary output. Human perception was not measured. None of 960 fixed Q4-generated payloads recovered using saved Q8 ranks for one Qwen revision. These results demonstrate artifact-specific surface-form concealment and configured inversion, with measured acceptability, transmission and detectability limitations. An observer with the full receiver configuration may decode, so surface concealment does not establish confidentiality.

V4 AUTHOR-REVIEW CANDIDATE

Introduction

Generative language models provide a new medium for text steganography because a prompt can define a next-token distribution and a topic direction in which generated text appears. In the Calgacus rank-transcoding construction [1], a sender tokenizes an arbitrary hidden text, records each token’s context-dependent rank, and generates a cover text by selecting the correspondingly ranked tokens under a shared cover prompt. The receiver reverses these steps under the same contexts. Calgacus thereby transcodes one hidden-text token into one cover-text token. RankCloak retains that rank transfer mechanism but also maps exact serialized cryptographic artifacts through direct subword and bounded codecs, for which payload and cover token counts need not equal the Calgacus baseline. This mechanism connects classical work on covert channels, information theoretic steganography, information hiding, and provably secure steganography [2–5] with neural and LLM-based linguistic steganography, where hidden information is embedded through choices made during language generation [6–14]. Related work on LLM watermarking also treats token selection as a carrier of hidden signals, although watermarking generally asks whether text carries a detectable mark rather than whether a chosen payload can be recovered [15]. Selective watermarking can restrict marking and detection to positions above an entropy threshold [16]. This provides a peer-reviewed precedent for selective marking. RankCloak defines and evaluates its own payload gate. The watermarking precedent supplies no guarantee of arbitrary-payload recovery. A long-running goal in this area is to make arbitrary, encrypted, or otherwise high-entropy data appear as ordinary communication. Early mimic-function work used statistical source models and inverse compression ideas to generate outputs with distributional properties similar to a target source [17, 18]. Hiding the Hidden and related linguistic-steganography systems transformed ciphertext into innocuous natural language text that could later be transformed back into the original ciphertext [19, 20]. Other text methods carried bits through synonym choice or contextual word substitution [21], while network camouflage systems generalized a similar objective to protocol formats by making encrypted traffic resemble allowed or innocuous traffic [22, 23]. RankCloak follows this same broad goal in an LLM setting. Instead of using a fixed grammar, synonym table, or protocol template, it uses the

25 token choices made by a shared language model. The payload is first converted into ranks, and the cover prompt determines which English tokens occupy those ranks at each step. This also makes the method useful as a measurement tool since it exposes how a hidden artifact changes rank pressure, generated text quality, and the traces available for later detection.

This measurement focus also has defensive value for detecting the presence of such hidden encodings. If hashes, 30 nonces, UUID-like values, or ciphertext strings are carried by generated tokens rather than displayed in their original syntax, surface scanners cannot rely only on visible hexadecimal, base64, UUID, or credential-shaped patterns. Detection must therefore look for indirect evidence, such as low-probability forced tokens, tokenizer artifacts, formatting fragments, unusually regular token choices, segmentation patterns, or differences between payload bearing spans and non-payload continuations. Covert-channel and stegomalware surveys emphasize that 35 hiding the existence of communication creates detection, capacity-limitation, and countermeasure problems across text and network media [24,25]. Text steganalysis and generated-text detection work similarly shows that statistical, neural, graph-based, and probability-based features can reveal some generated or steganographic text, while also showing that robust detection is setting-dependent and vulnerable to distribution shift or paraphrase [26–31]. The same issue is relevant to AI-safety monitoring, where hidden message passing and secret collusion among agents 40 have been studied as ways that generated text might evade oversight [32,33]. RankCloak evaluates these as distinct endpoints. Artifact concealment asks whether the cryptographic artifact and its recognizable syntax are directly visible in the cover text. Payload recovery asks whether an authorized receiver using the shared recovery configuration can reconstruct that artifact, whereas steganalysis asks whether an observer can classify the cover as likely encoded. A positive steganalysis decision does not itself display or recover the concealed artifact, although 45 high detectability weakens covert-channel stealth.

RankCloak studies synthetic hashes, authentication tags, nonces, identifiers, ciphertexts and signatures [34–36]. Direct subword tokenization can represent these strings compactly, but poorly predicted payload tokens can impose large contextual ranks [37,38]. We compare direct subword rank measurement with bounded ASCII-byte and hex-nibble codecs. Smaller rank alphabets constrain forced choices while increasing cover length. Segmentation 50 distributes ranks across prompt contexts and appends non-payload greedy continuations. Forced-span and full-message diagnostics remain separate because adding a continuation can raise the full-message average without changing any payload-bearing token.

The contribution is an artifact-specific representation and measurement framework for surface-form concealment and configured inversion. Saved-token replay fixes tokenization and history. Visible-text and transformed-text replay test 55 stronger transport assumptions. We evaluate these endpoints alongside detectability, entropy gating, quantization and intact-message contextual acceptability. Two local model judges assess whether meaning can be followed while allowing minor awkwardness. These automated scores do not measure human perception. A receiver configuration is a compatibility contract rather than a cryptographic secret.

Methods

60 RankCloak converts an exact serialized artifact into integer ranks, selects the correspondingly ranked tokens under a cover prompt, and recovers the ranks by replaying those tokens. Inverting the payload representation then reconstructs the original bytes. The non-segmented procedure carries one payload in one cover sequence. The segmented procedure distributes a hexadecimal payload across shorter forced spans and appends non-payload greedy continuations. Both retain separate rank, representation and serialized-byte endpoints.

65 Sender and receiver share a configuration K_{common} specifying the exact model revision, tokenizer, quantization and backend, prompt rendering, deterministic rank rule, codec, filter, span convention, and segmentation, tail or lead-in policy. This is a common protocol definition, not a formal cryptographic key. In the saved-token condition, retained token IDs hold tokenization and history fixed while the decoder recomputes contextual ranks and inverts the codec. Visible-text and transformed-text replay separately test delivery without that preservation.

70 The exact-copy contract requires one-indexed ranks with descending-logit and increasing-token-ID ordering. Byte codecs operate on the exact serialization, while hex-nibble coding accepts only canonical lowercase hexadecimal. Non-segmented decoding replays the complete saved cover sequence. Segmented decoding reads only the declared forced spans after replaying any lead-in, and ignores non-payload tails. Exact serialized recovery requires matching length, syntax, bytes and SHA-256 digest. Saved offsets, role masks and framing checks are detailed in

75 Supplementary Note S1, Replay modes and structured endpoints.

Basic rank definition

Rank-transcoding uses the rank of a token in an LLM next-token ordering as the transferable object. A rank is defined by sorting candidate tokens under a model context. Exact recovery is possible only when the sender and receiver replay the same model, tokenizer, prompt, rank rule, and generated tokens. Let M denote an autoregressive language model with vocabulary $\mathcal{V}$. For a token context h , let $z_M(v | h)$ be the next-token logit for token $v \in \mathcal{V}$, and let $\text{id}(v)$ be its deterministic token id. The model probability is:

$$p_M(v | h) = \frac{\exp z_M(v | h)}{\sum_{u \in \mathcal{V}} \exp z_M(u | h)}. \quad (1)$$

Ranks are one-indexed; token ID breaks ties deterministically:

$$u \prec_h v \iff z_M(u | h) > z_M(v | h) \text{ or } [z_M(u | h) = z_M(v | h) \text{ and } \text{id}(u) < \text{id}(v)]. \quad (2)$$

The rank of a token and the inverse rank selector are,

$$\rho_h(v) = 1 + |\{u \in \mathcal{V} : u \prec_h v\}|, \quad \text{token}_h(r) = v \text{ such that } \rho_h(v) = r. \quad (3)$$

Here $\text{token}_h(r)$ means the token at rank r in the deterministic ordering for context h . Because softmax is order-preserving, the released implementation computes this order directly from 64-bit logit values and breaks exact ties by increasing token ID.

Calgacus rank-transcoding baseline

The starting point is the Calgacus rank-transcoding construction, in which ranks rather than token identities are transferred between contexts [1]. In the Calgacus procedure, a hidden text can be tokenized as $e_1, e_2, \dots$. For each hidden-text token e_i , the sender computes its rank under a payload-side context g_i , where g_i is determined by the preceding hidden-text tokens and any payload-side conditioning used by the protocol. The sender then emits a cover token by selecting the token with the same rank under a cover-side context h_i , where h_i is determined by the cover prompt or key and the previously emitted cover tokens.

For hidden-text token e_i , payload-side context g_i , cover-side context h_i , emitted cover token y_i , and transferred rank r_i , the baseline rank-transcoding step is:

$$r_i = \rho_{g_i}(e_i), \quad y_i = \text{token}_{h_i}(r_i), \quad \hat{r}_i = \rho_{h_i}(y_i). \quad (4)$$

Under exact replay with the same model, tokenizer, prompt configuration, and deterministic rank rule, the recovered rank satisfies $\hat{r}_i = r_i$. The receiver can then replay the payload-side context and reconstruct the hidden-text token as $\hat{e}_i = \text{token}_{g_i}(\hat{r}_i)$. This is a same-token-length arbitrary-text baseline; one hidden-text token determines one cover token. The construction shows that a rank sequence can be carried by generated text without directly emitting the hidden tokens. Direct application to cryptographic-artifact strings, however, can impose high rank pressure. Such strings are often poorly predicted by a language model, so their payload-side tokens can occupy low-probability, high-rank positions, and selecting those same ranks under a cover prompt can degrade model-likelihood and surface diagnostics even when recovery remains exact.

RankCloak keeps the cover-side rank-transfer idea from Calgacus but changes the payload-side representation.

Instead of relying only on direct LLM tokenization of the displayed artifact, RankCloak maps serialized artifact strings into explicit bounded rank sequences. In the RankCloak notation used below, the displayed artifact is x , the payload codec is C , and the encoded rank sequence is $R = C(x)$. Recovery therefore proceeds by reconstructing $\hat{R}$ from the cover tokens and decoding $\hat{x} = C^{-1}(\hat{R})$, rather than by replaying a natural-language hidden-text stream token by token. The direct-subword condition retains the Calgacus payload-side ranks as a separate measurement arm. The contribution relative to the baseline is the measurement framework around artifact-specific representations: direct subword rank diagnostics, bounded ASCII-byte and hex-nibble codecs, deterministic token filtering, segmented forced spans, non-payload tails, and separate forced-span versus full-message metrics.

RankCloak bounded payload-codec methods

RankCloak changes the payload side by applying explicit codecs to displayed synthetic payload strings. The goal is to avoid treating a hash-like or ciphertext serialization as an ordinary natural-language token sequence. Instead, the displayed artifact is converted into a controlled sequence of ranks before cover generation begins. This makes rank pressure measurable by design. A codec can trade cover length against rank magnitude so that smaller rank

alphabets constrain forced token choices but require more generated tokens, while more compact representations can reduce length but may impose larger rank pressure. For payload string x , codec C , encoded rank sequence R , and recovered rank sequence $\hat{R}$,

$$R = C(x) = (r_1, \dots, r_n), \quad \hat{x} = C^{-1}(\hat{R}). \quad (5)$$

The direct-subword arm is not folded into Eq. (5); it tokenizes the literal serialization under the Calgacus payload context and retains the resulting ranks without imposing a fixed rank alphabet. The released implementation disables special-token insertion and accepts only a reversible tokenization whose detokenization differs from the literal UTF-8 payload by an explicitly recorded leading-space prefix. Any other affix or payload-byte change makes that representation unavailable before generation; complete framing metadata appears in Supplementary Note S1. Non-segmented cover generation and replay recovery then use the same rank selector as the Calgacus baseline, but the ranks come from the payload codec rather than only from direct payload-side LLM token ranks:

$$y_i = \text{token}_{h_i}(r_i), \quad \hat{r}_i = \rho_{h_i}(y_i), \quad \hat{R} = (\hat{r}_1, \dots, \hat{r}_n). \quad (6)$$

Codec-level exact recovery requires both rank recovery and an invertible codec for the displayed payload:

$$\hat{x} = x \quad \text{when} \quad \hat{R} = R \quad \text{and} \quad C^{-1}(C(x)) = x. \quad (7)$$

Hex-nibble coding applies to canonical lowercase hexadecimal payloads. It maps each displayed hexadecimal character to one bounded rank:

$$0 \mapsto 1, \quad 1 \mapsto 2, \quad \dots, \quad 9 \mapsto 10, \quad \mathbf{a} \mapsto 11, \quad \dots, \quad \mathbf{f} \mapsto 16. \quad (8)$$

Thus each lowercase hex character maps to one rank in $1, \dots, 16$. A 64-character SHA-256 hexadecimal serialization becomes 64 ranks under this codec. Base64 and hyphenated UUID display forms are rejected rather than silently normalized. The bounded alphabet controls maximum requested rank while increasing the number of forced positions; its relationship to observed cover diagnostics is evaluated rather than assumed. ASCII-byte fixed-radix coding applies to arbitrary displayed payload strings. It first expresses the displayed UTF-8 byte string as base- B digits d_i , then maps each digit to rank $d_i + 1$,

$$d_i \in \{0, \dots, B - 1\}, \quad r_i = d_i + 1. \quad (9)$$

The released codec concatenates the complete serialized byte bitstream, appends zero bits only at its end to reach a multiple of $\log_2 B$, and records the original byte length and terminal-padding count. Decoding subtracts one from each rank, validates the digit range, removes only that recorded padding, and reconstructs exactly the declared number of bytes. Consequently, the $B = 8$ condition uses $\lceil 8m/3 \rceil$ ranks for m serialized bytes rather than independently padding every byte; the $B = 16$ condition uses two ranks per byte. Supplementary Note S1 gives the complete metadata and edge cases.

When a deterministic token filter is configured, ranks are computed within the allowed token set A_h :

$$\rho_h^F(v) = 1 + |\{u \in A_h : u \prec_h v\}|, \quad A_h \subseteq \mathcal{V}. \quad (10)$$

Here A_h is the allowed token set after filtering. If no filter is configured, $A_h = \mathcal{V}$. The inverse selector $\text{token}_h^F(r)$ denotes the token at rank r in the same ordered allowed set. Sender and receiver must apply the identical mask and ordering because removing one candidate shifts every lower choice. A trial is unavailable if the allowed set cannot supply a requested rank.

Algorithm 1 gives the non-segmented bounded-codec RankCloak procedure. The algorithm first encodes the displayed artifact into ranks. It then generates one cover token per rank under the cover prompt. Recovery replays the same prompt and exact saved token sequence, recomputes each token's rank, and decodes the recovered ranks back to the displayed payload. This algorithm is the basic measurement unit for the non-segmented cover-generation trials.

Algorithm 1 Non-segmented RankCloak bounded-codec trial for encoding a displayed synthetic artifact as ranks, generating one cover token per rank, and recovering by exact-copy replay.

Require: Synthetic payload x ; model M ; cover prompt p ; payload codec C ; optional deterministic token filter F ; shared configuration K_{common} .

Ensure: Cover text, recovered payload $\hat{x}$, exact-recovery flag, and recorded cover metrics.

```

1:  $R \leftarrow C.\text{encode}(x)$ 
2:  $\text{context} \leftarrow \text{tokenize}(p)$ 
3:  $\text{cover\_tokens} \leftarrow []$ 
4: for each rank  $r_i \in R$  do
5:    $\text{logits} \leftarrow M.\text{next\_logits}(\text{context})$ 
6:   if  $F$  is configured then
7:      $\text{allowed\_tokens} \leftarrow F(\text{logits})$ 
8:   else
9:      $\text{allowed\_tokens} \leftarrow \text{vocabulary}(M)$ 
10:  end if
11:   $y_i \leftarrow \text{token with one-indexed rank } r_i \text{ among allowed\_tokens}$ 
12:  append  $y_i$  to  $\text{cover\_tokens}$ 
13:  append  $y_i$  to  $\text{context}$ 
14: end for
15:  $\text{cover\_text} \leftarrow \text{detokenize}(\text{cover\_tokens})$ 

16:  $\text{context} \leftarrow \text{tokenize}(p)$ 
17:  $\text{recovered\_ranks} \leftarrow []$ 
18: for each exact saved token  $y_i$  in  $\text{cover\_tokens}$  do
19:    $\text{logits} \leftarrow M.\text{next\_logits}(\text{context})$ 
20:   if  $F$  is configured then
21:      $\text{allowed\_tokens} \leftarrow F(\text{logits})$ 
22:   else
23:      $\text{allowed\_tokens} \leftarrow \text{vocabulary}(M)$ 
24:   end if
25:   append  $\text{rank\_of}(y_i, \text{logits}, \text{allowed\_tokens})$  to  $\text{recovered\_ranks}$ 
26:   append  $y_i$  to  $\text{context}$ 
27: end for
28:  $\hat{x} \leftarrow C.\text{decode}(\text{recovered\_ranks})$ 
29:  $\text{exact\_recovery} \leftarrow (\hat{x} = x)$ 
30: return  $\text{cover\_text}, \hat{x}, \text{exact\_recovery}$ 

```

Representations are validated before generation, and records retain the requested ranks, exact token IDs, configuration and framing metadata. The same loop accepts validated direct-subword ranks, with the payload-side reconstruction and structured endpoint checks in Supplementary Note S1.

Complete deterministic token filter specification

This specification documents the existing filter without changing its behavior. For `safe_text_filter_v1`, the following rules apply to each individually decoded vocabulary token.

Rule	Exact exclusion criterion
Empty or replacement Controls	Empty string or any U+FFFD replacement character. Any code point below 32 except newline and tab. Carriage return is excluded. DEL and other Unicode controls are not excluded by this rule alone.
Literal fragments	Any case insensitive occurrence of “‘, ‘, \section , \frac , {\, \rm , http, www., .com, <, >, &, </, <, >, [,], ___, .
Start rule	After stripping whitespace, a piece starting with # or ##. The second check is redundant.
Backslashes	Two consecutive backslashes or at least two backslashes anywhere in the piece.

Table 1. Complete exclusion rules of the retained safe-text filter. A piece is allowed only if none applies.

The filter decodes one token at a time with `safe_detokenize`. Byte output uses UTF-8 replacement decoding. If
 160 detokenization raises an exception, the wrapper returns an angle-bracket token marker, which the filter rejects. Any
 exception that reaches mask construction instead yields an empty piece, which is also rejected. Whitespace alone is
 allowed unless it contains an excluded control. Filtering does not guarantee that concatenated pieces are free of
 markup or tokenization changes.

The mask is cached by model object identity and filter name. No filter, the empty name, and `none` leave the
 165 vocabulary unmasked. Unknown names, missing vocabulary size, an entirely rejected vocabulary, incompatible mask
 length, unavailable requested ranks, and a target excluded by the mask raise errors. The selector validates positive
 one-indexed ranks. Masks are constructed with the model vocabulary length.

Candidate scores are converted to float64. The stated inverse ordering contract assumes finite logits. The historical
 helpers do not explicitly reject nonfinite scores. Selection sorts all allowed candidates by descending logit and
 170 ascending token ID. Recovery of a known token counts greater logits and equal logits at lower IDs. Both operations
 use the same allowed set. The isolated roundtrip filter adds a separate tokenization test. Ordinary sampling
 separately excludes BOS, EOS, and EOT identifiers where exposed by the backend. These exclusions are not
 implicit safe-text rules.

The [versioned token-filter source](#) is available in the repository. Its exact version and SHA-256 are retained in the V4
 175 filter audit.

RankCloak segmented multi-cover method

The segmented multi-cover method extends RankCloak from one cover sequence to several shorter public messages.
 Long sequences of forced ranks can accumulate topic drift, formatting artifacts, or low-probability continuations.
 Segmentation restarts the cover context for each chunk, allowing the effect of distributing the payload across several
 180 messages to be measured without claiming that those messages are human-natural. The method is also useful
 analytically because it separates the payload-bearing forced span from the rest of the public message.
 For a full rank sequence R , segmentation splits the payload into m chunks:

$$R = R^{(1)} \parallel R^{(2)} \parallel \dots \parallel R^{(m)}. \quad (11)$$

Here $\parallel$ denotes sequence concatenation, and each $R^{(j)}$ is an ordered, non-overlapping rank chunk. The completed
 design includes frozen segmented schedules and an ablation over 4-, 8-, 16-, and 32-rank chunks rather than treating
 185 one chunk size as universally preferred.

For segment j , the forced span emits one token per payload rank. Recovery uses only those forced tokens:

$$y_k^{(j)} = \text{token}_{h_k^{(j)}}(r_k^{(j)}), \quad \hat{r}_k^{(j)} = \rho_{h_k^{(j)}}(y_k^{(j)}). \quad (12)$$

When a filter is configured, both operations use the filtered ordering defined after Eq. (10). After the forced span
 $y^{(j)}$, the sender appends a greedy non-payload tail $z^{(j)}$. The public message is the tokenizer rendering of the
 concatenated token sequence $y^{(j)} \parallel z^{(j)}$. The receiver ignores $z^{(j)}$ and decodes only $y^{(j)}$:

$$m_j = \text{text}(y^{(j)} \parallel z^{(j)}), \quad \hat{R} = \hat{R}^{(1)} \parallel \dots \parallel \hat{R}^{(m)}. \quad (13)$$

190 Here $\text{text}(\cdot)$ means ordinary tokenizer rendering of the token sequence; it is not an additional cryptographic
 operation. The canonical expression in Eq. (13) has no lead-in. For the evaluated lead-in extension, a non-payload
 prefix is generated before $y^{(j)}$, and the record stores exact lead-in, forced, tail, and full token-ID arrays, a token-role
 mask, and half-open forced-span offsets. The decoder uses those declared offsets and does not infer a payload
 boundary from visible prose.

195 A core component of this method is the forced-span and tail separation. The payload is carried only by the forced
 span, while the tail is a greedy continuation whose effect on the complete public message is measured separately.
 This distinction prevents a misleading quality result, full-message diagnostics can change because of the non-payload
 tail even if the payload-bearing forced span remains low probability or exhibits surface artifacts. For that reason,
 the Results report forced-span metrics and full-message metrics separately.

200 Algorithm 2 describes the segmented multi-cover protocol. It encodes the hex-like payload, splits the rank sequence
 into short chunks, generates one forced span per chunk, appends the selected tail-policy output after each forced
 span, and recovers the payload by replaying only the forced spans. Each segment is its own replay component:
 generation starts from its declared prompt, and recovery reads the exact saved token IDs within its declared
 forced-span boundary.

Algorithm 2 Segmented multi-cover RankCloak trial for distributing a hex-nibble payload across payload-bearing forced spans with non-payload greedy continuations.

Require: Synthetic hex-like payload x ; model M ; hex-nibble codec C_{hex} ; segment size s ; prompt schedule $P = (p_1, \dots, p_m)$; deterministic token filter F ; tail policy T ; shared configuration K_{common} .

Ensure: Public messages ($\text{message}_1, \dots, \text{message}_m$), recovered payload $\hat{x}$, exact-recovery flag, forced-span metrics, and full-message metrics.

```

1:  $R \leftarrow C_{\text{hex}}.\text{encode}(x)$ 
2: split  $R$  into chunks  $(R_1, \dots, R_m)$ , each with at most  $s$  ranks
3: for each segment  $j = 1, \dots, m$  do
4:   context  $\leftarrow \text{tokenize}(P[j])$ 
5:   forced_tokens $_j \leftarrow []$ 
6:   for each rank  $r \in R_j$  do
7:     logits  $\leftarrow M.\text{next\_logits}(\text{context})$ 
8:     allowed_tokens  $\leftarrow F(\text{logits})$ 
9:      $y \leftarrow$  token with one-indexed rank  $r$  among allowed_tokens
10:    append  $y$  to forced_tokens $_j$ 
11:    append  $y$  to context
12:   end for
13:   tail_tokens $_j \leftarrow \text{TAILPOLICY}(M, \text{context}, F, T)$ 
14:   message $_j \leftarrow \text{detokenize}(\text{concat}(\text{forced\_tokens}_j, \text{tail\_tokens}_j))$ 
15: end for
16: emit public messages ( $\text{message}_1, \dots, \text{message}_m$ )

17: recovered_chunks  $\leftarrow []$ 
18: for each segment  $j = 1, \dots, m$  do
19:   context  $\leftarrow \text{tokenize}(P[j])$ 
20:   read the exact saved tokens within the declared forced-span boundary for  $R_j$ 
21:   ignore all tail tokens
22:   segment_ranks  $\leftarrow []$ 
23:   for each forced token  $y$  in the payload-bearing span do
24:     logits  $\leftarrow M.\text{next\_logits}(\text{context})$ 
25:     allowed_tokens  $\leftarrow F(\text{logits})$ 
26:     append rank_of( $y, \text{logits}, \text{allowed\_tokens}$ ) to segment_ranks
27:     append  $y$  to context
28:   end for
29:   append segment_ranks to recovered_chunks
30: end for
31:  $\hat{x} \leftarrow C_{\text{hex}}.\text{decode}(\text{concat}(\text{recovered\_chunks}))$ 
32: exact_recovery  $\leftarrow (\hat{x} = x)$ 
33: return ( $\text{message}_1, \dots, \text{message}_m$ ),  $\hat{x}$ , exact_recovery

```

205 The non-lead-in form in Algorithm 2 supplies the canonical segment layout. The lead-in extension inserts a rank-1 prefix before the forced span; its exact token IDs must be replayed before forced-rank recovery, and its saved length shifts the half-open forced boundary. Lead-in lengths 2, 4, 8, 16, and 32 were compared with the no-lead-in condition. Single-topic segmentation reuses its assigned prompt, whereas multi-topic segmentation rotates deterministically through the declared prompt categories. Segment order, prompts, boundaries, and token roles are retained metadata rather than information inferred by the decoder.

210 Greedy tails allow a forced span to continue toward a sentence ending without carrying additional payload. The primary dynamic policy checks for terminal punctuation after at least eight tail tokens, requires balanced delimiters and no dangling connective or punctuation, and records a 256-token emergency cap as censoring. The fixed 20–60-token policy and no-tail condition are ablations. These are stopping heuristics, not judgments of semantic completeness. Supplementary Note S1, Segment records, lead-ins, and tail policies, gives the full procedure and Algorithm S1.

Evaluation corpus, controls, and metrics

The experiments use deterministic synthetic payloads that resemble cryptographic or operational artifacts while avoiding real secret material. The corpus contains 480 public deterministic test vectors, 60 in each of eight algorithm-defined classes; SHA-256 hexadecimal digests, HMAC-SHA256 hexadecimal tags, deterministic 96-bit nonces, 128-bit hexadecimal tokens, UUIDv4 identifiers, AES-256-GCM base64 outputs, ChaCha20-Poly1305 base64 outputs, and Ed25519 base64 signatures [34–36, 39–42]. Authenticated-encryption and signature rows are genuine outputs of the declared algorithms under reproducible public test inputs. No real credentials, API keys, private keys, account identifiers, operational secrets, commands, private user data, or deployed traffic are used.

Each payload is evaluated first as a representation problem and then, where applicable, as a cover-generation problem. The representation diagnostics measure how the exact serialization behaves under direct subword tokenization and how many ranks are required by the bounded codecs. Cover-generation conditions include direct subword ranks, ASCII-byte fixed-radix coding with $B = 8$, ASCII-byte fixed-radix coding with $B = 16$, raw hex-nibble coding for eligible lowercase hexadecimal payloads, and single- and multi-topic segmented hex-nibble protocols. All generation, recovery, and diagnostic rows use the deterministic rank-ordering rule in Eqs. (2) and (3) as part of the shared replay configuration.

Generation used Meta-Llama-3-8B-Instruct, Qwen2.5-7B-Instruct, and Mistral-7B-Instruct-v0.3 in pinned Q4_K_M configurations, an approximately 4-bit quantized format, with their embedded GGUF tokenizers. Eighteen English templates span six prompt categories. Secondary Spanish and Simplified Chinese conditions evaluate saved-token recovery but do not establish multilingual human naturalness. The primary matrix contains 14,400 generated units; 6,480 RankCloak payload trials and 7,920 prompt, model, representation, and length-matched ordinary controls. Complete corpus construction, model and tokenizer identities and hashes, prompt schedules, and work-unit accounting appear in Supplementary Note S2.

The protocol variants define how encoded ranks become public text. Non-segmented trials use Algorithm 1 and replay the complete saved cover-token sequence. Segmented trials use Algorithm 2 with a declared tail policy (Supplementary Note S1, Algorithm S1), while recovery uses only the saved forced tokens. Prompt-matched controls provide comparison text for automated diagnostics and detector training, they are not human-written naturalness baselines. The deterministic safe-text filter rejects declared control, replacement, code, markup, URL, escape, and formatting fragments. The ablation compares it with no filter and an isolated roundtrip-stable filter, lead-in lengths 0, 2, 4, 8, 16, and 32, chunk sizes 4, 8, 16, and 32, dynamic, fixed, and no-tail policies, and frozen single- versus multi-topic schedules.

The Results report exact serialized-payload recovery, representation and rank diagnostics, cover length, requested-rank pressure, token log probability, deterministic surface artifacts, and automated readability diagnostics. A balanced 504-stimulus subset covers seven generation conditions and all 18 English templates, its surface measures include Flesch Reading Ease [43], frozen artifact flags, and TF-IDF prompt similarity, with intervals bootstrapped over prompt templates. Source-model and held-out-evaluator likelihoods are reported as model diagnostics, not participant ratings. No human study was performed.

For segmented rows, every cover-quality measurement is assigned to one of two scopes. Forced-span metrics are computed only over payload-bearing tokens decoded by the receiver. Full-message metrics are computed over the complete rendered message, including non-payload tails and any experimental lead-in. This scope distinction is necessary because tails can change full-message proxies while carrying no payload ranks. Declared unavailable combinations are excluded from the applicable estimand and reported separately rather than classified as observed failures. Complete metric definitions, transformation coverage, ablation schedules, and exclusion rules appear in Supplementary Notes S3 through S6.

For paired segmented trials, tail gain is the full-message mean token log probability minus the forced-span mean token log probability, $G_{\text{tail}} = \bar{\ell}_{\text{full}} - \bar{\ell}_{\text{forced}}$. Positive values mean that averaging in the non-payload continuation raises the full-message model score; they do not imply a change in the payload-bearing span. The empirical frontier reports forced-span and full-message views separately on the four-class common hexadecimal support, with payload-bootstrap uncertainty and automated surface-flag burden detailed in Supplementary Note S5.

Recovery, robustness, and capacity endpoints

The saved-token endpoint supplies the retained payload-bearing token sequence to the decoder under K_{common} , thereby testing exact contextual rank reconstruction and codec inversion while tokenization fidelity and generation history are held fixed; retaining IDs does not make those inverse computations unnecessary or expose the payload serialization. Visible-text replay instead detokenizes and retokenizes the rendered string, and controlled transformed-text tests additionally apply final-tail truncation, Unicode NFKC, quote conversion, whitespace and

line-ending changes, deterministic character and token edits, markdown copy/paste, paraphrase, limited canonicalization, or cross-model decoding [44]. These are distinct transmission conditions: token-preserving delivery can retain the exact token stream, ordinary rendered-text delivery does not, and paraphrased delivery changes the visible string. Final-tail truncation removes only declared non-payload tokens and is not evidence of arbitrary truncation tolerance; first-divergence labels localize the first recorded mismatch but do not establish its cause. For a bounded representation containing H source bits and a rank alphabet of size B , the minimum fixed-radix forced length and nominal forced-position rate are

$$n_B = \left\lceil \frac{H}{\log_2 B} \right\rceil, \quad R_B = \frac{H}{n_B} \leq \log_2 B. \quad (14)$$

Here H is the declared input bit width of that codec, eight bits per exact serialized byte for the byte codecs and four bits per canonical hexadecimal character for the nibble codec. For the byte bitstream, $n_B = \lceil 8m / \log_2 B \rceil$, consistent with Eq. (9). Direct subword ranks do not have a fixed B and are excluded from this bounded-capacity identity. For representation comparisons on a common artifact support, we additionally report effective artifact bits per forced token, $R_{\text{art}} = H_{\text{art}} / n_{\text{forced}}$. Here H_{art} is held fixed across representations, four bits per canonical hexadecimal character in the common-hex analysis, rather than set to the codec’s serialized input width. This descriptive rate permits direct and bounded representations to be compared on the same artifacts; it is not a cryptographic-security capacity.

If segment j has ℓ_j lead-in tokens and t_j tail tokens, and $\delta \geq 0$ denotes any other counted non-payload extension under the retained accounting convention, full cover length and effective rate are,

$$N_{\text{full}} = n_{\text{forced}} + \sum_{j=1}^J (\ell_j + t_j) + \delta, \quad R_{\text{eff}} = \frac{H}{N_{\text{full}}} \leq \frac{H}{n_{\text{forced}}} = R_B \quad (n_{\text{forced}} = n_B). \quad (15)$$

Segmentation changes context resets and auxiliary text, not the number of forced codec positions. Positive cover-length residuals are overhead rather than payload capacity.

For eligible-token probabilities $p_{(1)}(h) \geq \dots \geq p_{(B)}(h)$, any bounded selection $r \leq B$ obeys the same-context conditional quality relation

$$-\log p_{(1)}(h) \leq -\log p_{(r)}(h) \leq -\log p_{(B)}(h), \quad Q_B = \frac{1}{n_B} \sum_{i=1}^{n_B} -\log p_M(y_i | h_i). \quad (16)$$

Some tables report mean log probability $\bar{\ell} = -Q_B$, for which larger values are less negative. These bounds concern model likelihood under identical filtered histories; they are not statements about grammar, meaning, human naturalness, security, or detectability.

The confirmatory endpoint is exact equality between the decoded serialized payload bytes and the declared source, including length, syntax, and SHA-256 digest. Exact rank and representation equality are retained as diagnostics. Trial-level Wilson 95% intervals are reported [45], a strict payload group passes only if every constituent prompt, model, and eligible protocol trial succeeds, and a group defined by model and payload applies the same rule within one model. Segments are never counted as independent payload trials. Grouped bootstrap procedures use payload or prompt template as the resampling unit according to the estimand. Declared unavailable combinations never enter a failure denominator.

Neural steganalysis, statistical analysis, and computational overhead

Corpus preparation preceded feature extraction and tokenization. One normalized duplicate pair was removed from 15,840 source rows, leaving 15,838 observations. Payload groups connected by exact or declared near-duplicate text formed immutable components assigned to 9,904 training, 2,952 validation and 2,982 test observations. No audited payload, pair, text, component or near-duplicate link crossed partitions. This safeguard applies to the declared normalization and similarity rules, not all semantic paraphrases. Supplementary Note S11 gives the normalization, thresholds and split audit.

TextCNN and a pinned DeBERTa-v3-base revision were trained specifically on RankCloak versus matched ordinary model controls using the predeclared architectures. An exact-model-aware logistic detector, representing an attacker with likelihood information from the same model that generated the text, used summary surprisal features computed from saved token log probabilities for both labels, with regularization selected by validation ROC-AUC; ordinary-control ranks were unavailable and therefore excluded. ‘Leave one family out’ fits excluded the target

model family from both training and validation. A secondary test selected 250 human-written responses from the Databricks Dolly corpus across 16 available templates without using human labels for fitting or threshold selection, it is a corpus comparison, not a participant study or naturalness rating.

ROC-AUC used tie-aware empirical ranks. Low false-positive-rate thresholds were selected on validation data, frozen before testing and reported only when both negative sample sizes supported the requested resolution. Intervals used 2,000 resamples of complete deduplication clusters (Supplementary Note S12).

The entropy extension fixed ungated, median and 75th-percentile strict thresholds from ordinary top- p development traces. Payload ranks were consumed only at eligible positions. These historical trajectories used no token filter and ordinary sampled skips. Their completion-conditional serialized rate counts eight bits per payload byte, not Shannon channel capacity. The quantization comparison held one Qwen revision and the remaining generation contract fixed while comparing Q4_K_M with Q8_0. Supplementary Notes S13–S15 give the gate calibration, paired tests, detector transfer and provenance.

Statistical inference respects payload and prompt template grouping, with segments nested within covers. Paired ablations use payload as the pairing and bootstrap unit. Mixed models include payload and prompt grouping where supported, negative-binomial models for overdispersed artifact counts, and Gaussian models for continuous outcomes [46, 47]. Holm adjustment controls selected contrast families; Benjamini-Hochberg values accompany exploratory families. Convergence, dispersion, zero-inflation, singularity, and residual diagnostics are retained. Three of five fitted mixed models were singular, no undeclared fixed-effects fallback was substituted, and compact ablation, topic, auxiliary detector, and near duplicate analyses remain labeled exploratory or post outcome where applicable. Complete model formulas, bootstrap seeds, split definitions, diagnostics, and multiplicity details are in Supplementary Notes S5 through S7.

Generation overhead uses inclusive wrapper wall time retained by the harness, including the reported setup, generation, and supported replay scopes. Payload throughput is representation bits divided by generation time, and effective rate follows Eq. (15). Detector benchmarks report fixed-device CUDA wall/training time and peak allocated VRAM. CPU time, repeated warm-up measurements, and cross-device hardware generalization were unavailable and were not inferred. Complete timing cells and component scopes appear in Supplementary Note S9.

Intact-message contextual acceptability

The primary coherence evaluation asks whether a complete delivered message is understandable in its authentic prompt context, allowing minor awkwardness. Each message is shown intact without marking its payload boundary or inventing preceding dialogue. Rotating-topic covers are assessed separately. This automated follow-up was designed after inspecting the fragment results and frozen before its own scoring. Structured model evaluation motivates an explicit rubric [48], but does not replace direct human assessment [49].

A seeded identity hash selects 48 payloads, 12 per hexadecimal class, outside the earlier 92-payload scored cohort. All three generators and both schedules are retained, with two covers per trial, giving 288 trials and 576 messages. Selection uses neither text quality nor the old window-alignment rules. The sample represents this outside-cohort stratum, not all 240 original payloads. Supplementary Note S20 specifies inclusion probabilities and selection details.

Each encoded message is paired with ordinary output from the same pinned generator and authentic prompt. The control replaces the eight forced positions with greedy choices and uses the same safe-text filter, greedy continuation, assigned 264-token cap and heuristic stopping rule. Natural stops and cap-censored outputs remain intact. Each item is judged independently by the other two model families, with identical instructions and anonymous presentation. Judges see no payload, condition label, generator identity, boundary or previous score. A separate constructed calibration set checks rubric interpretation before the final freeze.

Disruption levels 0 and 1 count as acceptable, allowing minor grammar errors, repetition or unusual but interpretable wording. Levels 2 and 3 denote material disruption or unintelligibility. Secondary logical connectedness runs from 1 to 5 and is not another acceptance gate. The panel weights both judges equally, averages messages within trial and trials within payload, and uses 2,000 shared class-stratified payload-cluster bootstrap draws for paired intervals. Complete-pair estimates require both judges in both conditions and condition on the fixed, reused greedy controls. This compares delivered-message conditions, not an isolated boundary effect. Supplementary Note S20 gives the full rubric, settings, calibration and missingness analysis.

Decoder compatibility and bounded transport diagnostics

The canonical configuration fingerprint includes model and tokenizer identities, quantization and backend, prompt rendering, BOS/EOS policy, ordering, filter, codec/framing, gate, and boundary/reset contract. The new compatibility API accepts a matching canonical header and rejects incomplete, corrupted or mismatched

configurations. Tests change each component independently. This is a compatibility check, not authentication, error correction or proof of confidentiality. It was not retroactively added to the historical covers.

The transport diagnostic selects at most one original visible-text success and failure per source model by identity hash. An audit found that the historical raw “unmodified” reference points to saved-token recovery. Visible baseline labels therefore come from executed historical text-transform replays only when every observed byte equals the source text. The historical references and composite Markdown outcomes remain unchanged. Missing strata are unavailable. Five selected covers support at most 25 decomposed arms. We add only missing model-input replays for per-line prefix insertion, outer trimming, trailing trimming, line-ending normalization and removal of the declared blockquote wrapper. Wrapper removal strips exactly the declared prefix and cannot inspect payloads or search for a successful offset. Source bytes, retokenized IDs, saved-offset spans, recovered ranks and bytes are retained in Supplementary Note S18.

Revision assistance

OpenAI Codex assisted with code development, retained-data analysis, validation and drafting of the revision.

Reported computational results trace to the pinned model runs and retained records. The author is responsible for reviewing these analyses and the final text.

Results

Evaluation design and exact-copy recovery

The completed primary design produced 14,400 work units across three model families and 18 English prompt templates. Of these 6,480 were payload bearing RankCloak trials spanning the six protocol/codec configurations and 7,920 were ordinary controls. All 6,480 decoded serialized payloads exactly under saved-token replay (1.000000; Wilson 95% CI [0.999408, 1.000000]). The stricter aggregation also passed for all 480 payload groups (CI [0.992061, 1.000000]) and all 1,440 groups defined by model and payload (CI [0.997339, 1.000000]). These grouped endpoints prevent repeated prompts and protocols from inflating the apparent number of independently generated artifacts.

The recovery question was whether codec inversion remained exact after forcing the ranks into cover contexts, rather than whether emitted ranks could merely be read back from the same loop. Success was defined at the original serialization and digest, and every model and eligible protocol stratum reached that endpoint. The all success result therefore supports deterministic implementation fidelity across the evaluated prompt/model matrix. Its uncertainty is one sided in substance, the data bound the rate of error under this completed design but cannot show that an unobserved replay environment or transmission channel would also be error-free.

Secondary exact-copy tests recovered 288/288 Spanish trials and 288/288 Simplified Chinese trials, or 576/576 in total. They establish recovery for those retained prompts under the same saved-token contract; they do not establish multilingual naturalness or robustness to ordinary text transfer. Recovery strata and secondary-language endpoints appear in Supplementary Tables S5 and S6.

Payload representation, rank pressure, and capacity tradeoff

Figure 1 compares payload representations across 480 unique artifacts in eight classes and three models. The 1,440 direct observations across payloads and models contained 60,861 direct rank positions. Direct subword representations required 11 to 77 forced symbols, with class medians ranging from 20 to 66, and the largest observed direct rank was 145,498. These counts and ranks depend on the model and tokenizer rather than defining a fixed alphabet. The deterministic codecs instead imposed rank ceilings of 8 or 16 while generally increasing forced length. On the four-class common hexadecimal support, median effective artifact rates were approximately 4.571 for direct subword, 4.000 for hex nibble, 2.000 for ASCII $B = 16$, and 1.497 for ASCII $B = 8$ artifact bits per forced token. These descriptive rates compare representations on identical eligible artifacts; they are not cryptographic-security capacities. The whiskers in Fig. 1 are observed 5th to 95th percentiles, not confidence intervals. The detailed theoretical capacity and tail validation tests a different question of whether retained records satisfy the declared algebraic identities and conditional bounds (in Supplementary Fig. S6 and Table S14). The common support empirical capacity and quality frontier follows in Fig. 2.

Payload representation trade-off

Whiskers: observed 5th-95th percentiles; x: observed maxima

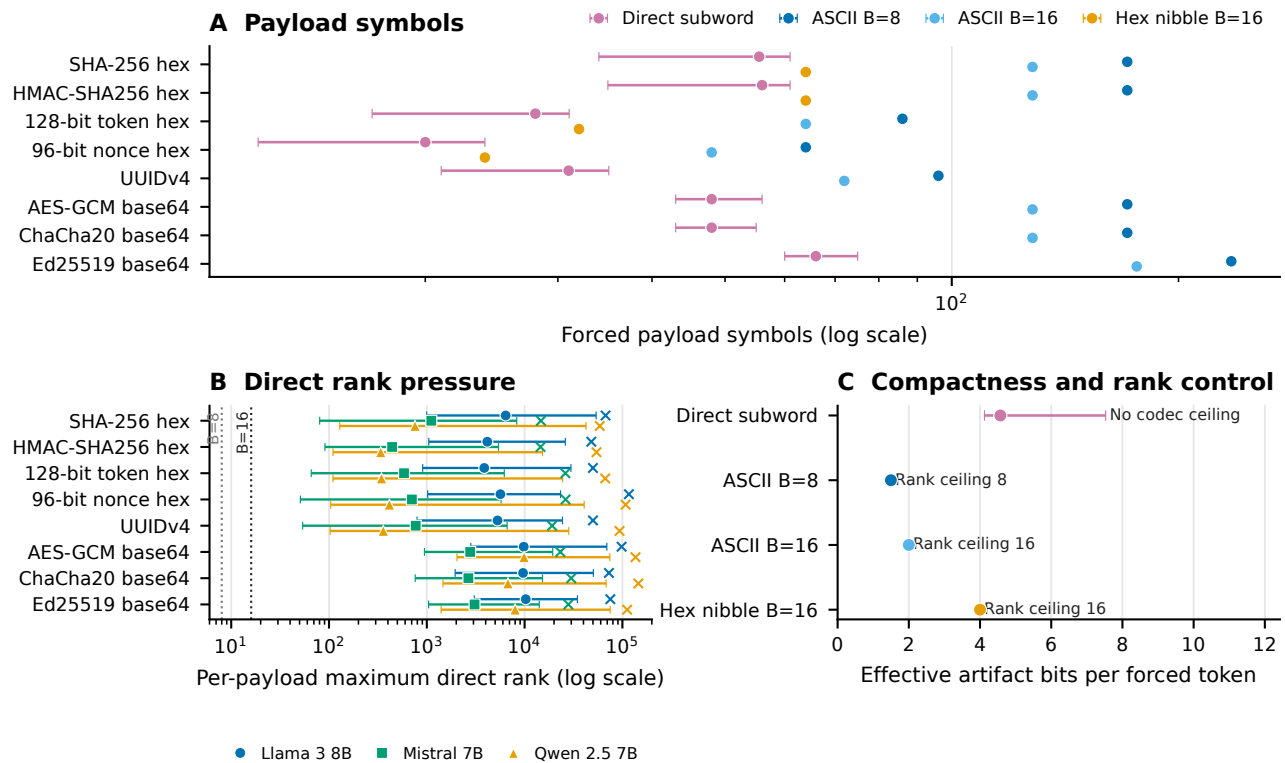


Figure 1. Payload representation, rank pressure, and capacity tradeoff. Direct-subword symbol counts and rank pressure are summarized for 1,440 observations across payloads and models comprising 60,861 direct rank positions. Their values are tokenizer and model dependent. Bounded codecs constrain requested ranks to at most 8 or 16, at the cost of more forced positions. Effective artifact-rate comparisons use the four class common hexadecimal support and are descriptive rather than cryptographic-security capacities. Whiskers show observed 5th to 95th percentiles, not confidence intervals.

Figure 2 compares mean token count with mean source model token log probability over 4,320 trials on the four class common hexadecimal support. Three models and six protocols form 18 model and protocol cells, and separate forced span and full message views give 36 plotted cells. Compact direct subword spans generally occupy the low length region but can have poorer source model likelihood, whereas bounded protocols trade longer forced spans for controlled rank alphabets. Segmented full message points move toward better full message likelihood by adding non-payload continuation length, this movement is not improved likelihood on the payload-bearing forced span. Higher mean token log probability is better, point area represents mean automated surface flag burden, and uncertainty uses 2,000 resample payload-bootstrap 95% confidence intervals. The complete primary matrix contains no lead-in condition, so exploratory lead-in rows were not pooled. These automated diagnostics are not human-perceived judgments, and the empirical frontier is distinct from the theoretical capacity and tail validation.

Empirical capacity-quality frontier

Bars: payload-bootstrap 95% CIs; point area: mean surface flags

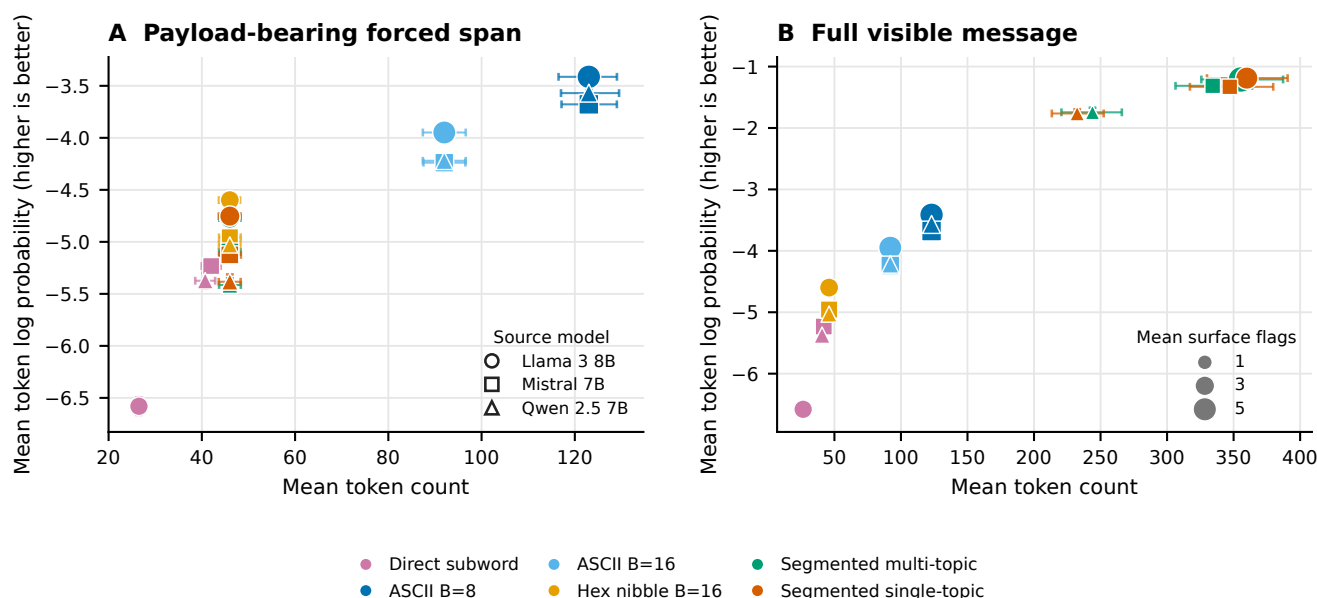


Figure 2. Empirical capacity and quality frontier. Mean token count is shown against mean source-model token log probability for 4,320 trials on four-class common hexadecimal support, spanning three models, six protocols, 18 model and protocol cells, and 36 forced-span or full message cells. Higher mean token log probability is better; point area represents mean automated surface-flag burden. Error bars are 2,000 resample payload-bootstrap 95% confidence intervals. The primary matrix contains no leadin condition. Likelihood and surface flags are automated diagnostics, not human judgments, and this empirical frontier is distinct from the theoretical capacity and tail validation in Supplementary Fig. S6.

Exact-copy replay and transmission fragility

425 Replay mode materially changed recovery (Fig. 3). Saved token replay recovered 144/144 robustness covers, whereas detokenizing the same covers and retokenizing their visible text recovered 88/144 (0.611111). So even an unedited visible string did not guarantee the original token sequence. The historical raw unmodified row reports 144/144 by reference to saved-token recovery and is not another executed visible-text trial. Common transformations were damaging. Unicode NFKC recovered 82/144 (0.569444), quote conversion 56/144 (0.388889), and whitespace

430 trimming 28/144 (0.194444). Markdown copy/paste and paraphrase recovered 0/144, and the two cross-model decoder outcomes for each cover recovered 0/288.

Deleting only the declared final 10% of a message recovered 144/144 because that operation was constrained to non-payload tail tokens. It is not evidence of arbitrary truncation tolerance, removing a forced token or earlier context would change the decoded sequence. The broader matrix also showed low recovery after token or character

435 edits and line-ending conversion (Supplementary Note S4). The limited-canonicalization unmodified row also references saved-token recovery. Its executed transformed cells all failed, so they did not demonstrate a general mitigation.

Exact-copy fragility

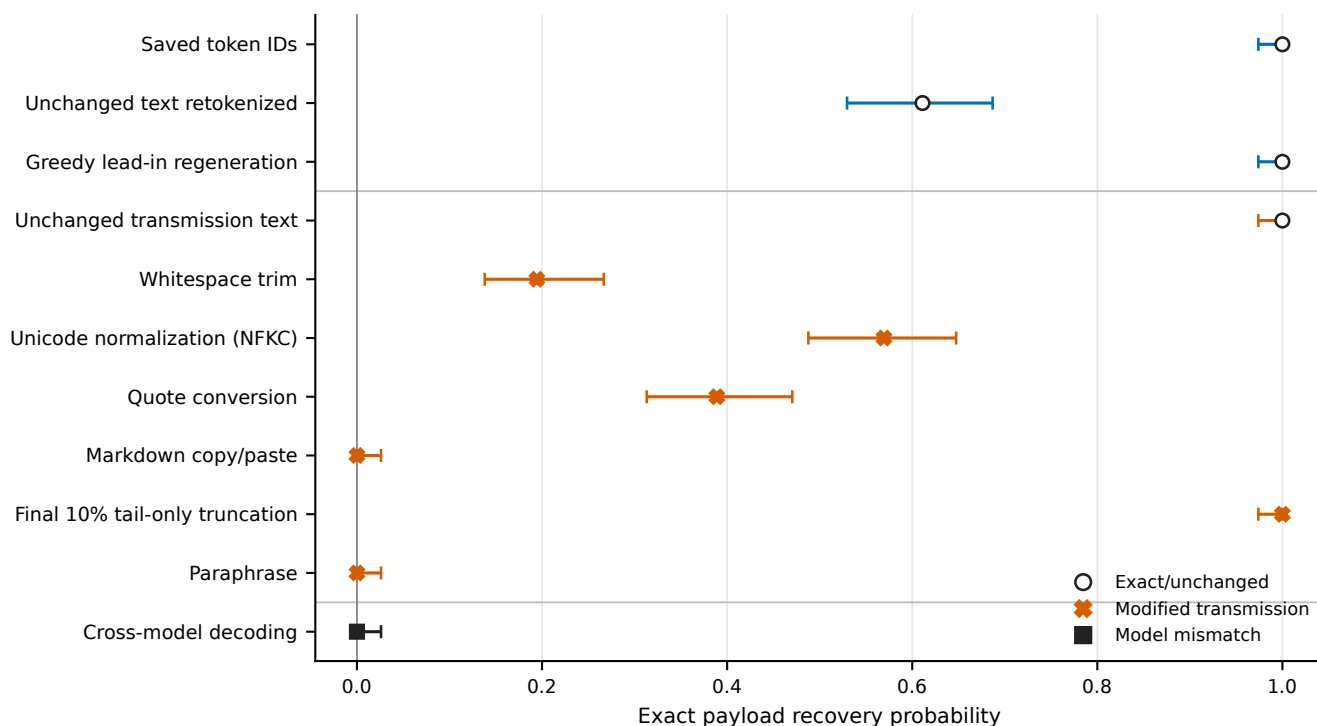


Figure 3. Exact-copy fragility. Recovery proportions and Wilson 95% intervals are shown for the principal replay and transmission conditions. Saved token IDs preserve the strongest replay contract. The plotted unchanged transmission text is the historical saved-ID reference, not another visible-text replay. Visible text detokenization and retokenization already reduce recovery. NFKC normalization, quote conversion, and trimming reduce it further, while the tested composite Markdown operation, paraphrase, and cross-model decoding yield no successful payloads. Final 10% tail truncation removes only declared non payload tail tokens and must not be interpreted as arbitrary truncation tolerance. The full transformation matrix, denominators, limited canonicalization cells, and first divergence diagnostics appear in Supplementary Figure S1 and Supplementary Note S4.

Forced span versus full message

Across 1,440 paired segmented trials, overall forced-span mean token log probability was -5.090904 and overall full message mean token log probability was -1.424774, for an overall paired gain of +3.666130 (Supplementary Fig. S17). Each of the six model and topic schedule cells contained 240 paired trials, and all six 2,000 resample paired payload bootstrap confidence intervals for the gain excluded zero.

Non-payload greedy continuations contain no payload ranks and add no payload capacity. The higher full message average therefore reflects the inclusion of non payload greedy continuations and must not be interpreted as improved quality on the payload bearing forced span. Automated readability and surface diagnostics remain in Supplementary Fig. S2 and Note S5; they are not human evaluation.

Ablations, dynamic stopping, and prompt-topic effects

The locked compact ablation extraction compared 48 paired payload groups across three models and is exploratory/post-outcome (Supplementary Notes S6 and S9). A 32-token lead-in, relative to no lead-in, reduced mean log probability by 1.652327 (95% CI [-1.749091, -1.553103]) and added 159.590278 full-message tokens (CI [125.829861, 191.030035]). This adverse result is consistent with added context and boundary cost, the contrast does not prove a single failure mechanism. Increasing the segment size from 8 to 32 ranks increased effective rate by 1.231461 bits per full token (CI [1.073116, 1.388188]) and reduced full length by 174.500000 tokens (CI [-208.405035, -140.573264]), trading fewer tails for longer forced spans. The lead-in and segment results locate different costs. A leadin precedes the forced span, consumes visible tokens, and changes the distribution used for every encoded rank; it therefore cannot be treated as a harmless stylistic prefix. Larger segments keep the forced count fixed but require fewer prompt resets and tail opportunities. Their higher effective rate is consistent with reduced overhead, while the forced span itself lasts longer before an ordinary greedy continuation begins.

Removing the safe-text filter changed mean log probability by approximately -0.0018 (canonical CSV estimate -0.001797; CI [-0.037738, 0.035394]) and effective rate by -0.062969 bits per full token (CI [-0.134265, 0.007651]), both intervals included zero. The Mistral roundtrip stable filter cell was unavailable for 48 work units, although the no-filter contrast included all three models. The filter results therefore do not establish a quality benefit. Compared with dynamic stopping, no tail increased effective rate by 3.171551 bits per full token and shortened messages by 254.951389 tokens. This arithmetic consequence of removing non-payload text is not evidence of better cover quality. A fixed sentence-tail policy of 20 to 60 tokens shortened messages by 65.708333 tokens relative to dynamic stopping, whereas its selected effective-rate interval included zero. Dynamic stopping remains a declared heuristic with an emergency cap, not a semantic judgment by readers.

Across 15 locked topic contrasts, one Llama artifact-count comparison excluded the null; multi-topic versus single-topic expected count ratio 1.276361 (95% CI [1.114743, 1.461409], Holm-adjusted $p = 0.003294$). The other 14 adjusted p -values were at least 0.396248. This compact extraction was performed post outcome without refitting models and is exploratory as it does not support a general prompt-topic advantage.

Neural steganalysis

On the strictly deduplicated matched test, TextCNN ROC-AUC was 0.9868 (95% CI [0.9819, 0.9913]), DeBERTa-v3-base was 0.9994 (CI [0.9984, 1.0000]), and the exact-model-aware surprisal attacker was 0.999995 (CI [0.999970, 1.000000]; Fig. 4). Their normalized empirical 0–1% partial AUCs were 0.6652, 0.9948, and 0.999460. These values show that the encoding is often detectable; classification of a cover does not reveal the concealed artifact or restore its recognizable surface syntax.

Leave one family out ROC-AUC ranged from 0.9186 to 0.9784 for TextCNN, 0.9972 to 1.0000 for DeBERTa, and 0.999943 to 1.000000 for the model aware detector. These fits exclude the target family from training and validation and test unseen family transfer within the three-family corpus, not arbitrary future models. At validation frozen nominal 1% FPR thresholds, matched test TPR/observed FPR was 0.8317/0.0127 (19/1,491 false positives), 0.9913/0.0007 (1/1,491), and 0.9987/0.0020 (3/1,491), respectively. The matched validation and test sizes support empirical 0.1% reporting, at that target the corresponding TPRs were 0.4829, 0.9886, and 0.9987, with 2, 1, and 3 observed false positives. Family holdouts had only 497 test negatives and did not support 0.1% estimates; at frozen 1% thresholds their observed false-positive counts reached 35, 28, and 3, showing calibration drift.

Leave-one-model-family-out detector generalization

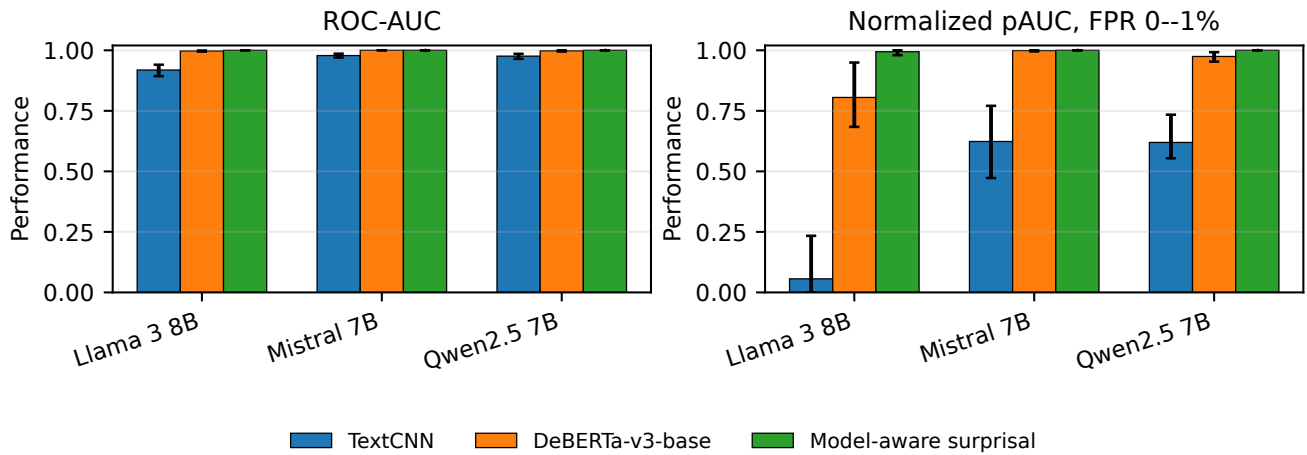


Figure 4. Strictly deduplicated detector evaluation. Matched and leave-one-family-out ROC-AUC results are shown for two data adaptive text-only detectors and the exact-model-aware surprisal attacker. Higher performance indicates that encoded covers can be distinguished from matched controls. Components connected by declared exact or near-duplicate relations do not cross partitions, the rule does not detect semantic paraphrases below its threshold. Supplementary Note S12 reports full split counts, validation-frozen low-FPR operating points, exact false-positive counts, and unavailable 0.1% conditions.

Human controls, entropy gating, and quantization

The bounded human-authored secondary analysis selected 250 Dolly responses across 16 available templates, with no exact or declared near-duplicate links within that corpus or to the primary corpus. Against these negatives, ROC-AUC was 0.7537 for TextCNN and 0.9846 for DeBERTa. At thresholds selected only on matched

ordinary-model validation controls, 152/250 and 97/250 human texts were false positives. These high rates reveal detector domain sensitivity and may reflect generic ‘machine versus human’ differences, the analysis is not a participant study, human naturalness rating, or evidence that readers cannot notice RankCloak covers.

Entropy gating produced a tradeoff among payload rate, length and detectability (Table 2). From ungated through median and strict gates, mean serialized-payload rate conditional on completion declined from 7.4001 to 5.7920 and 4.0513 bits/token, while mean length ratio increased from 1.0000 to 1.3743 and 2.7615. The strict condition completed 114/120 payloads, its six maximum-budget failures are retained outcomes, and its rate denominator is the 114 completed cases. Text only ROC-AUC declined under the strict gate to 0.8850 and 0.8893, but exact-model-aware ROC-AUC remained 0.9904. Visible-text recovery was 62/120, 65/120, and 68/120 across the three conditions, so gating neither established undetectability nor reliable visible text transport.

For the single pinned Qwen revision, saved-ID recovery was 960/960 for both Q4_K_M and Q8_0, whereas visible text recovery was 732/960 (76.25%) and 758/960 (78.96%). The paired Q8-minus-Q4 difference was 2.71 percentage points (grouped-bootstrap 95% CI $[-0.73, 6.35]$), paired outcomes were 588 both, 144 Q4 only, 170 Q8 only, and 58 neither, with exact two-sided McNemar $p = 0.1582$. Thus there was no statistically supported Q8 recovery advantage. Independently generated paths diverged substantially (mean positionwise match 0.0155), while cross-quantization detection remained very strong in both directions.

Analysis or condition	Recovery or completion	Payload rate and length	Detection or paired inference
Strict detector split	15,838 retained rows; 9,904/2,952/2,982 train/validation/test	Deduplication before features; zero declared cross-partition leakage	Matched AUC 0.9868/0.9994/0.999995
Human-authored controls	250 frozen test negatives; no human labels used for fitting	Length-matched within 16 available templates	TextCNN/DeBERTa AUC 0.7537/0.9846; false positives 152/250 and 97/250
Entropy ungated	Complete/saved ID 120/120; visible 62/120	7.4001 bit/token, $n = 120$, conditional on completion; length ratio 1.0000	AUC TextCNN/DeBERTa/model-aware 0.9953/0.9999/1.0000
Entropy median gate	Complete/saved ID 120/120; visible 65/120	5.7920 bit/token, $n = 120$, conditional on completion; length ratio 1.3743	AUC 0.9825/0.9785/0.9999
Entropy strict gate	Complete/saved ID 114/120; visible 68/120; six budget failures	4.0513 bit/token, $n = 114$, conditional on completion; length ratio 2.7615	AUC 0.8850/0.8893/0.9904
Q4_K_M versus Q8_0	Saved ID 960/960 each; visible 732/960 versus 758/960	Independent-path position match 0.0155	Difference 2.71 points, CI $[-0.73, 6.35]$; McNemar $p = 0.1582$
Cross-quantization detection	Q4-to-Q8 and Q8-to-Q4 held-payload tests	One Qwen revision; two quantizations	AUC TextCNN 0.9927/0.9944; DeBERTa and model-aware 1.0000/1.0000

Table 2. Summary of detector, entropy and quantization analyses. The entropy rate counts eight bits per serialized payload byte per generated token and is conditional on completion. It is not a Shannon channel capacity. Entropy recovery proportions use all 120 attempts. Detector triples are TextCNN, DeBERTa-v3-base, and exact model aware surprisal. Empirical 0.1% operating points are unavailable for the 250-negative human comparison and every entropy and quantization subgroup because each has fewer than 1,000 negatives. Detailed denominators, intervals, thresholds, and condition-level results appear in Supplementary Notes S11-S15.

Contextual acceptability of complete messages

All three judges passed the constructed calibration check while retaining two missed contradictions across Llama and Mistral. Supplementary Note S20 reports the complete calibration without treating it as human validation.

The full frozen sample contains 576 encoded messages from 288 trials and 48 payloads, including 85 messages unavailable under the old window rules. Using 537 complete pairs, the equally weighted panel accepts 90.3 [88.4, 92.1]% of encoded messages and 96.6 [95.0, 98.1]% of matched greedy ordinary messages. The paired difference is -6.3 [-8.7, -3.9] percentage points. Brackets denote 95% payload-cluster intervals conditional on paired availability. The Mistral-source difference interval includes zero, which does not establish equivalence. Table 3 and Fig. 5 show the overall and generator-specific estimates.

Source	Pairs	RankCloak % [95% CI]	Ordinary % [95% CI]	Difference pp [95% CI]	Disagreement Enc.%	Ord.%
Panel overall	537	90.3 [88.4, 92.1]	96.6 [95.0, 98.1]	-6.3 [-8.7, -3.9]	14.6	6.7
Llama	182	91.1 [87.0, 94.8]	100.0 [100.0, 100.0]	-8.9 [-13.0, -5.2]	7.3	0.0
Mistral	191	87.8 [84.1, 91.2]	92.7 [88.8, 96.1]	-4.9 [-9.9, 0.3]	23.4	14.6
Qwen	164	92.3 [89.1, 95.1]	98.1 [95.9, 99.7]	-5.9 [-9.1, -2.7]	13.0	3.7

Table 3. Intact-message contextual acceptability. Acceptance is disruption level 0 or 1. The equally weighted two-judge panel is averaged within trial and payload. The selected sample contains 48 payload clusters, with 288 trials overall and 96 per generator. Pairs counts complete panel-paired messages. Paired percentile intervals use 2,000 class-stratified payload draws and condition on the fixed greedy controls. The Qwen row has 47 observed payload clusters. Disagreement uses the same hierarchy on available pairs of judges within each arm, with 562 encoded and 551 ordinary messages overall. These are automated ratings, not human reading outcomes.

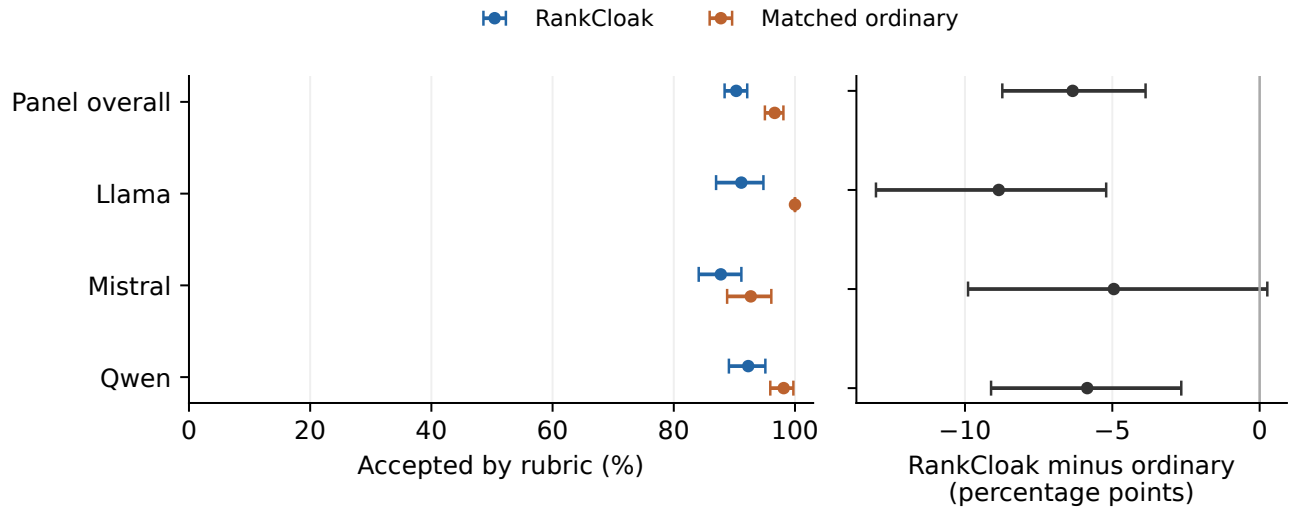


Figure 5. Automated acceptability of intact messages. The two-judge panel applies the same lenient rubric to RankCloak and matched greedy ordinary output. Left, acceptance at disruption levels 0 or 1. Right, the paired difference in percentage points. Error bars are 95% intervals from 2,000 class-stratified payload-cluster draws. Intervals condition on the fixed control set. Generator rows use different pairs of judges, so generator and judge effects are not fully crossed. These are automated judgments, not human acceptance or detection rates.

Mean logical connectedness is 3.60 [3.53, 3.67]/5 for encoded messages and 4.00 [3.94, 4.06]/5 for ordinary messages. Binary judge disagreement is 14.6% and 6.7%, respectively. Most accepted encoded ratings indicate minor awkwardness rather than completely smooth meaning. The 537 complete pairs span 280 trials and all 48 payloads. The 39 incomplete pairs are retained in the missingness analysis in Supplementary Note S20, alongside four-level distributions, judge and schedule estimates, lengths and sensitivity bounds.

Judges' literal disruption quotations and identity-selected full-message examples are reported in Supplementary Note S20, including unfavorable cases. Boundary mapping after scoring is descriptive, not causal localization. The earlier fragment pilots and limited fixed-path likelihood findings remain complementary evidence in Supplementary Note S17.

Mechanisms of transmission failure

The 25 decomposed transport arms across five identity-selected covers required 26 missing segment replays and 53 context-capacity checks of reused historical inputs. All reused ranks and error outcomes agreed at the new allocation capacity. These 79 distinct segment inputs remain within the same 25 logical arms. Prefix-only insertion recovered

none of the five covers. All 27 selected segment instances presented a prohibited first forced token and were rejected before rank recovery. They do not measure a downstream rank avalanche. Declared wrapper removal recovered one originally failing Mistral case but did not supply a general repair. The original 0/144 composite-Markdown endpoint remains unchanged. Byte, span, token and rank evidence is retained in Supplementary Note S18.

Strict-gate failure contexts

The six strict-gate failures all use ASCII B16 without a token filter. Five are Qwen trials and one is Mistral. They consume 59 to 116 of 128 requested ranks, or 54 of 64, at their original budgets. Longest below-threshold runs range from 35 to 267 positions. Matched ungated and moderate runs complete, while successful strict runs can also contain extended low-entropy stretches. The aligned text windows and progress evidence describe associations without identifying a causal linguistic trigger (Supplementary Note S16).

Fixed-message cross-quantization decoding

The quantization audit separates 56,321 changed observed ranks at 122,220 encoded positions from 13,207 at 122,220 ordinary positions. A distinct offline Q4-to-Q8 fixed-message endpoint recovers zero of 960 original payloads. Of these trials, 766 contain invalid bounded ranks and 194 decode valid but incorrect bytes. This uses saved Q8 ranks on the historical Q4 token path and establishes no reverse-direction result. For identical histories and allowed sets, a logit gap greater than twice a uniform absolute perturbation bound preserves a pairwise ordering. This sufficient condition does not estimate the actual quantization error. Independent free-generation divergence introduces different histories, whereas observed-token replay continues to feed received cover tokens, not recovered payload symbols.

Computational overhead and generalizability

Across the 18 primary model and protocol cells, mean inclusive generation time ranged from 1.674368 to 16.087628 seconds. Mean payload throughput ranged from 13.631855 to 100.851772 bits/s, and effective rate from 0.710004 to 7.322003 bits per full token. These saved wall times include wrapper setup and decoding scope defined by the harness, CPU time and repeated warm-up measurements were not retained.

The approximately order of magnitude spread across cells shows that representation and model choice materially affect operational cost in the retained environment. Compact direct ranks can reduce forced work, whereas byte expansion and repeated segmented continuations increase it. Throughput and effective rate answer different questions: the first normalizes payload bits by wall time, while the second normalizes by visible full-message tokens. Neither is a deployment benchmark outside the recorded wrapper and device. On the fixed CUDA benchmark, representative TextCNN wall/training times were 91.358/9.498 seconds with 306.25 MB peak allocated VRAM, DeBERTa-v3-base times were 1,536.661/1,425.063 seconds with 6.38 GB. Those device-specific measurements do not generalize across hardware. Evaluation across three model families and exact-copy Spanish and Simplified Chinese trials broadens the controlled design.

The implementation separates sorting from rank recovery. For vocabulary size V and allowed-set size A , selecting a requested rank performs a full stable sort in $O(A \log A)$, plus $O(V)$ array and mask work. Recovering a known token's rank counts greater scores and lower-ID ties in $O(A)$. Full-vocabulary probabilities and entropy require additional vocabulary passes. Greedy lead-ins, tails, diagnostic rank-one choices and rank- B quality checks also invoke sorting, so one sort per payload symbol understates the executed work.

The inspected arithmetic-coding implementation sorts before truncation and then constructs and scans finite-precision cumulative intervals [8]. SAAC also sorts the full vocabulary before adapting its retained set [10]. ADG sorts, searches and removes candidates while forming groups, then recurses in the selected group [11]. Its cost depends on the data structures used for removal and mass updates. All methods additionally incur their language-model calls. RankCloak segmentation repeats prefills, direct-subword transcoding adds payload-side model passes, and skips and tails consume work without adding payload digits. Memory includes weights, KV state, vocabulary temporaries and $O(CV)$ score storage for context limit C with `logits_all=True`. Supplementary Note S19 gives the implementation-specific comparison. These bounds do not imply a wall-time advantage across different backends.

Discussion

RankCloak provides a reproducible method for converting serialized cryptographic artifacts into language-model next-token ranks and realizing those ranks as cover text. Its representation choices expose a central tradeoff that direct subwords can be compact but leave rank pressure uncontrolled, whereas ASCII-byte and hex-nibble codecs

bound the permitted rank by spending more forced positions. Segmentation redistributes those positions across contexts, and non-payload greedy continuations add non-payload length. The formal and empirical capacity relationships make these costs explicit rather than treating longer cover text as additional payload capacity. The principal demonstrated transformation is that the recognizable surface representation of a synthetic cryptographic artifact can be absent from the generated cover while an authorized receiver reconstructs it under the supported shared configuration. In the saved-token condition, retaining the token stream isolates tokenization fidelity, but recovery still requires contextual rank reconstruction and codec inversion; ordinary visible-text transfer is the stricter and materially less reliable condition. Steganalysis addresses another endpoint: a detector can often flag the presence of encoding, weakening covert-channel stealth, without thereby observing the original artifact or recovering its serialization.

The intact-message follow-up measures a practical but explicitly automated endpoint. Under the lenient rubric, the panel accepts 90.3% of encoded messages compared with 96.6% of matched greedy controls. This quantifies acceptance despite minor awkwardness while retaining material disruptions and judge disagreements. It is neither a human reading rate nor evidence of indistinguishability. The comparison concerns complete delivered conditions, and length or content differences can contribute to the observed contrast.

The protocol comparisons explain part of the efficiency tradeoff. Larger segments reduce repeated tail overhead, while lead-ins change every subsequent forced-token history and worsened likelihood and length in the selected contrasts. A higher full-message likelihood can arise simply by adding greedy continuation tokens. It does not improve the forced span or increase its payload capacity.

Strict pre-feature component deduplication found zero leakage under the declared exact and character- n -gram rules, yet matched, unseen-family, and exact-model-aware detectors remained strong. Validation-frozen low-FPR results show that high ranking performance does not ensure threshold calibration after family or quantization shift. The human-authored comparison is informative but secondary: its high false-positive rates expose control-domain sensitivity and may reflect general machine-versus-human cues.

Entropy gating reduced text-only detectability at the cost of lower serialized-payload rates, longer covers and six strict-gate failures, while exact-model-aware detection remained strong. Quantization likewise preserved within-configuration saved-ID inversion without providing a statistically supported Q8 advantage in visible-text recovery. Together, the studies identify where configured artifact inversion succeeds and where transmission, efficiency and detection constrain its use. Historical conditional context gain adds evidence about local fixed-path compatibility, distinct from the primary intact-message acceptability endpoint.

Configuration sensitivity, detection and a proposed fallback

Small configuration changes have no universal recovery rate. A tokenizer update may preserve a string's tokenization or split and merge tokens, changing saved span offsets and later conditioning. Weight or quantization changes can reorder candidates even on an identical received history. A gate disagreement changes which positions consume payload digits. The conditional rank-agreement argument in Supplementary Note S19 makes these dependencies explicit without assigning a recovery rate from a version label.

The evaluated model-aware detector is trace informed. It consumes summaries of retained untempered full-vocabulary token log probabilities. Reproducing those features requires the exact model and numerical backend, original token IDs, prompts, boundaries and reset schedule. Scoring only visible text after retokenization is a distinct, unvalidated access condition. The ordinary sampler instead uses temperature 0.8 and top- p 0.95. Payload-induced rank choices need not follow that sampler, and bounded ranks do not enforce distribution matching. These plausible detection mechanisms are not causal attributions. Supplementary Note S19 states the support conditions for distribution comparisons. AUC is not a KL estimate. An observer possessing the complete receiver configuration may also decode the payload. Surface-form concealment establishes neither confidentiality nor indistinguishability against this attacker.

A possible future fallback would identify a supported configuration, frame independently decodable blocks with explicit resets and integrity checks, apply bounded outer error or erasure correction, and request retransmission when repair fails. For minimum distance d , a conventional code requires $2e + s < d$ for unique correction of e errors and s located erasures under its channel assumptions. Reliable boundaries and a correct decoder codebook remain prerequisites. ECC cannot repair an arbitrary wrong model codebook or persistent synchronization loss, and retransmitting under the same unresolved mismatch does not solve it. Metadata, redundancy, checks and retries consume length and compute and may add detector cues. This proposal is not implemented and supplies no new recovery result. Supplementary Note S19 provides the conditional arguments and operation counts.

Responsible use and limitations

RankCloak can encode sensitive looking artifacts, but it does not itself provide encryption, authentication, confidentiality or a cryptographic security proof. Protected payloads require an external cryptographic system and an explicit threat model. Perfect primary recovery was demonstrated under the preserved-token replay contract. Visible-text transmission and common transformations are fragile, and the tested perturbations do not establish broad edit robustness. Human-perceived naturalness, broad multilingual behavior, real-world deployment and performance across arbitrary future models or communication channels were not established. The contextual judges are small local instruction models, not human participants. Their generator assignments are not fully crossed, and reused greedy controls form a fixed reference set. The selected payloads exclude the earlier scored cohort. Acceptance under this rubric neither proves indistinguishability nor isolates a causal effect of an encoding boundary. The historical fragment assays and their unsuccessful semantic control checks are retained in Supplementary Note S17. Detector performance is limited to the evaluated corpus. Detailed boundaries appear in Supplementary Table S17 and Notes S16 through S20.

Data Availability

The source code and retained computational data are available at <https://github.com/mantzaris/llm-rankcloak>. Published version 2.0.0, DOI <https://doi.org/10.5281/zenodo.22555497>, covers V3 and does not include the V4 revision evidence. The concept DOI is <https://doi.org/10.5281/zenodo.21987449>. The new contextual acceptability study and revised documents are retained in the repository. The earlier local archive candidate predates this study and does not include it. A new verified archive covering the current evidence and a public version DOI remain requirements before journal submission.

Code Availability

The custom generation, replay and analysis code accompanies these data in the same repository. The published V3 version, earlier local candidate and current unarchived follow-up are distinguished above.

References

1. Norelli, A. & Bronstein, M. M. LLMs can hide text in other text of the same length. In *The Fourteenth International Conference on Learning Representations* (2026).
2. Simmons, G. J. The prisoners' problem and the subliminal channel. In *Advances in Cryptology: Proceedings of CRYPTO '83*, 51–67, DOI: [10.1007/978-1-4684-4730-9_5](https://doi.org/10.1007/978-1-4684-4730-9_5) (Springer, 1984).
3. Cachin, C. An information-theoretic model for steganography. In *Information Hiding*, 306–318, DOI: [10.1007/3-540-49380-8_21](https://doi.org/10.1007/3-540-49380-8_21) (Springer, 1998).
4. Hopper, N. J., Langford, J. & von Ahn, L. Provably secure steganography. In *Advances in Cryptology – CRYPTO 2002*, 77–92, DOI: [10.1007/3-540-45708-9_6](https://doi.org/10.1007/3-540-45708-9_6) (Springer, 2002).
5. Petitcolas, F. A. P., Anderson, R. J. & Kuhn, M. G. Information hiding: A survey. *Proc. IEEE* **87**, 1062–1078, DOI: [10.1109/5.771065](https://doi.org/10.1109/5.771065) (1999).
6. Fang, T., Jaggi, M. & Argyraki, K. Generating steganographic text with LSTMs. In *Proceedings of ACL 2017, Student Research Workshop*, 100–106, DOI: [10.18653/v1/P17-3017](https://doi.org/10.18653/v1/P17-3017) (Association for Computational Linguistics, 2017).
7. Yang, Z.-L., Guo, X.-Q., Chen, Z.-M., Huang, Y.-F. & Zhang, Y.-J. RNN-Stega: Linguistic steganography based on recurrent neural networks. *IEEE Transactions on Inf. Forensics Secur.* **14**, 1280–1295, DOI: [10.1109/TIFS.2018.2871746](https://doi.org/10.1109/TIFS.2018.2871746) (2019).
8. Ziegler, Z. M., Deng, Y. & Rush, A. M. Neural linguistic steganography. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, 1210–1215, DOI: [10.18653/v1/D19-1115](https://doi.org/10.18653/v1/D19-1115) (Association for Computational Linguistics, 2019).
9. Dai, F. Z. & Cai, Z. Towards near-imperceptible steganographic text. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 4303–4308, DOI: [10.18653/v1/P19-1422](https://doi.org/10.18653/v1/P19-1422) (Association for Computational Linguistics, 2019).

- 680 10. Shen, J., Ji, H. & Han, J. Near-imperceptible neural linguistic steganography via self-adjusting arithmetic coding. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, 303–313, DOI: [10.18653/v1/2020.emnlp-main.22](https://doi.org/10.18653/v1/2020.emnlp-main.22) (Association for Computational Linguistics, 2020).
11. Zhang, S., Yang, Z., Yang, J. & Huang, Y. Provably secure generative linguistic steganography. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, 3046–3055, DOI: [10.18653/v1/2021.findings-acl.268](https://doi.org/10.18653/v1/2021.findings-acl.268) (Association for Computational Linguistics, 2021).
- 685 12. Wang, Y., Song, R., Li, L., Zhang, R. & Liu, J. Dynamically allocated interval-based generative linguistic steganography with roulette wheel. *Appl. Soft Comput.* **176**, 113101, DOI: [10.1016/j.asoc.2025.113101](https://doi.org/10.1016/j.asoc.2025.113101) (2025).
13. Wu, J., Wu, Z., Xue, Y., Wen, J. & Peng, W. Generative text steganography with large language model. In *Proceedings of the 32nd ACM International Conference on Multimedia*, 10345–10353, DOI: [10.1145/3664647.3680562](https://doi.org/10.1145/3664647.3680562) (Association for Computing Machinery, 2024).
- 690 14. Lin, K., Luo, Y., Zhang, Z. & Luo, P. Zero-shot generative linguistic steganography. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 5168–5182, DOI: [10.18653/v1/2024.naacl-long.289](https://doi.org/10.18653/v1/2024.naacl-long.289) (Association for Computational Linguistics, 2024).
- 695 15. Kirchenbauer, J. *et al.* A watermark for large language models. In *Proceedings of the 40th International Conference on Machine Learning*, vol. 202 of *Proceedings of Machine Learning Research*, 17061–17084 (PMLR, 2023).
16. Lee, T. *et al.* Who wrote this code? watermarking for code generation. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 4890–4911, DOI: [10.18653/v1/2024.acl-long.268](https://doi.org/10.18653/v1/2024.acl-long.268) (Association for Computational Linguistics, Bangkok, Thailand, 2024).
- 700 17. Wayner, P. Mimic functions. *Cryptologia* **16**, 193–214, DOI: [10.1080/0161-119291866883](https://doi.org/10.1080/0161-119291866883) (1992).
18. Wayner, P. Strong theoretical steganography. *Cryptologia* **19**, 285–299, DOI: [10.1080/0161-119591883962](https://doi.org/10.1080/0161-119591883962) (1995).
19. Chapman, M. & Davida, G. I. Hiding the hidden: A software system for concealing ciphertext as innocuous text. In *Information and Communications Security: First International Conference, ICICS 1997*, vol. 1334 of *Lecture Notes in Computer Science*, 335–345, DOI: [10.1007/BFb0028489](https://doi.org/10.1007/BFb0028489) (Springer, 1997).
- 705 20. Bennett, K. Linguistic steganography: Survey, analysis, and robustness concerns for hiding information in text. Tech. Rep. CERIAS TR 2004-13, Purdue University, Center for Education and Research in Information Assurance and Security (2004).
- 710 21. Chang, C.-Y. & Clark, S. Practical linguistic steganography using contextual synonym substitution and a novel vertex coding method. *Comput. Linguist.* **40**, 403–448, DOI: [10.1162/COLI_a_00176](https://doi.org/10.1162/COLI_a_00176) (2014).
22. Weinberg, Z. *et al.* StegoTorus: A camouflage proxy for the Tor anonymity system. In *Proceedings of the 2012 ACM Conference on Computer and Communications Security*, 109–120, DOI: [10.1145/2382196.2382211](https://doi.org/10.1145/2382196.2382211) (Association for Computing Machinery, 2012).
- 715 23. Dyer, K. P., Coull, S. E., Ristenpart, T. & Shrimpton, T. Protocol misidentification made easy with format-transforming encryption. In *Proceedings of the 2013 ACM SIGSAC Conference on Computer and Communications Security*, 61–72, DOI: [10.1145/2508859.2516657](https://doi.org/10.1145/2508859.2516657) (Association for Computing Machinery, 2013).
24. Zander, S., Armitage, G. & Branch, P. A survey of covert channels and countermeasures in computer network protocols. *IEEE Commun. Surv. Tutorials* **9**, 44–57, DOI: [10.1109/COMST.2007.4317620](https://doi.org/10.1109/COMST.2007.4317620) (2007).
- 720 25. Badar, L. T., Carminati, B. & Ferrari, E. A comprehensive survey on stegomalware detection in digital media, research challenges and future directions. *Signal Process.* **231**, 109888, DOI: [10.1016/j.sigpro.2025.109888](https://doi.org/10.1016/j.sigpro.2025.109888) (2025).
26. Wen, J., Zhou, X., Zhong, P. & Xue, Y. Convolutional neural network based text steganalysis. *IEEE Signal Process. Lett.* **26**, 460–464, DOI: [10.1109/LSP.2019.2895286](https://doi.org/10.1109/LSP.2019.2895286) (2019).
- 725 27. Wu, H., Yi, B., Ding, F., Feng, G. & Zhang, X. Linguistic steganalysis with graph neural networks. *IEEE Signal Process. Lett.* **28**, 558–562, DOI: [10.1109/LSP.2021.3062233](https://doi.org/10.1109/LSP.2021.3062233) (2021).
28. Wang, H., Yang, Z., Yang, J., Chen, C. & Huang, Y. Linguistic steganalysis in few-shot scenario. *IEEE Transactions on Inf. Forensics Secur.* **18**, 4870–4882, DOI: [10.1109/TIFS.2023.3298210](https://doi.org/10.1109/TIFS.2023.3298210) (2023).

29. Gehrmann, S., Strobelt, H. & Rush, A. GLTR: Statistical detection and visualization of generated text. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, 111–116, DOI: [10.18653/v1/P19-3019](https://doi.org/10.18653/v1/P19-3019) (Association for Computational Linguistics, 2019).
30. Mitchell, E., Lee, Y., Khazatsky, A., Manning, C. D. & Finn, C. DetectGPT: Zero-shot machine-generated text detection using probability curvature. In *Proceedings of the 40th International Conference on Machine Learning*, vol. 202 of *Proceedings of Machine Learning Research*, 24950–24962 (PMLR, 2023).
31. Sadasivan, V. S., Kumar, A., Balasubramanian, S., Wang, W. & Feizi, S. Can AI-generated text be reliably detected? stress testing AI text detectors under various attacks. *Transactions on Mach. Learn. Res.* (2025).
32. Motwani, S. R. *et al.* Secret collusion among AI agents: Multi-agent deception via steganography. In *Advances in Neural Information Processing Systems*, vol. 37, 73439–73486, DOI: [10.52202/079017-2336](https://doi.org/10.52202/079017-2336) (Curran Associates, Inc., 2024).
33. Zolkowski, A., Nishimura-Gasparian, K., McCarthy, R., Zimmermann, R. S. & Lindner, D. Early signs of steganographic capabilities in frontier LLMs. In *The Fourteenth International Conference on Learning Representations* (2026).
34. National Institute of Standards and Technology. Secure hash standard (SHS). Federal Information Processing Standards Publication 180-4, National Institute of Standards and Technology (2015). DOI: [10.6028/NIST.FIPS.180-4](https://doi.org/10.6028/NIST.FIPS.180-4).
35. Josefsson, S. The base16, base32, and base64 data encodings. RFC 4648, DOI: [10.17487/RFC4648](https://doi.org/10.17487/RFC4648) (2006).
36. Davis, K., Peabody, B. & Leach, P. Universally unique IDentifiers (UUIDs). RFC 9562, DOI: [10.17487/RFC9562](https://doi.org/10.17487/RFC9562) (2024).
37. Sennrich, R., Haddow, B. & Birch, A. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*, 1715–1725, DOI: [10.18653/v1/P16-1162](https://doi.org/10.18653/v1/P16-1162) (Association for Computational Linguistics, 2016).
38. Kudo, T. & Richardson, J. SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 66–71, DOI: [10.18653/v1/D18-2012](https://doi.org/10.18653/v1/D18-2012) (Association for Computational Linguistics, 2018).
39. Krawczyk, H., Bellare, M. & Canetti, R. HMAC: Keyed-hashing for message authentication. RFC 2104, DOI: [10.17487/RFC2104](https://doi.org/10.17487/RFC2104) (1997).
40. Dworkin, M. Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC. Special Publication 800-38D, National Institute of Standards and Technology (2007). DOI: [10.6028/NIST.SP.800-38D](https://doi.org/10.6028/NIST.SP.800-38D).
41. Nir, Y. & Langley, A. ChaCha20 and Poly1305 for IETF protocols. RFC 8439, DOI: [10.17487/RFC8439](https://doi.org/10.17487/RFC8439) (2018).
42. Josefsson, S. & Liusvaara, I. Edwards-curve digital signature algorithm (EdDSA). RFC 8032, DOI: [10.17487/RFC8032](https://doi.org/10.17487/RFC8032) (2017).
43. Flesch, R. A new readability yardstick. *J. Appl. Psychol.* **32**, 221–233, DOI: [10.1037/h0057532](https://doi.org/10.1037/h0057532) (1948).
44. Yan, R. & Murawaki, Y. Addressing tokenization inconsistency in steganography and watermarking based on large language models. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 7076–7098, DOI: [10.18653/v1/2025.emnlp-main.361](https://doi.org/10.18653/v1/2025.emnlp-main.361) (Association for Computational Linguistics, 2025).
45. Wilson, E. B. Probable inference, the law of succession, and statistical inference. *J. Am. Stat. Assoc.* **22**, 209–212, DOI: [10.1080/01621459.1927.10502953](https://doi.org/10.1080/01621459.1927.10502953) (1927).
46. Bates, D., Mächler, M., Bolker, B. & Walker, S. Fitting linear mixed-effects models using lme4. *J. Stat. Softw.* **67**, 1–48, DOI: [10.18637/jss.v067.i01](https://doi.org/10.18637/jss.v067.i01) (2015).
47. Brooks, M. E. *et al.* glmmTMB balances speed and flexibility among packages for zero-inflated generalized linear mixed modeling. *The R J.* **9**, 378–400, DOI: [10.32614/RJ-2017-066](https://doi.org/10.32614/RJ-2017-066) (2017).
48. Liu, Y. *et al.* G-eval: NLG evaluation using GPT-4 with better human alignment. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2511–2522, DOI: [10.18653/v1/2023.emnlp-main.153](https://doi.org/10.18653/v1/2023.emnlp-main.153) (Association for Computational Linguistics, 2023).

- 780 49. van der Lee, C., Gatt, A., van Miltenburg, E., Wubben, S. & Krahmer, E. Best practices for the human evaluation of automatically generated text. In *Proceedings of the 12th International Conference on Natural Language Generation*, 355–368, DOI: [10.18653/v1/W19-8643](https://doi.org/10.18653/v1/W19-8643) (Association for Computational Linguistics, 2019).

Acknowledgements

NA

Author contributions

A.V.M. is the sole author and is responsible for the study and the final manuscript.

785 Funding

NA

Additional Information

Supplementary Information accompanies this manuscript. Correspondence should be addressed to A.V.M.

Competing interests

790 The author declares no competing interests.